

Nano4 GP1 生醫 HPC 課程

第二堂：Slurm 與我的 skill

第 03 章 Slurm 語法與作業調度

第 04 章 AI 輔助生醫質控管線

📖 線上講義：<https://gemini960114.github.io/Nano4-Docs/>

今天的時間表

時間	內容
0:00-0:10	複習、重新連線
0:10-0:20	暖身：沒有規則檔時，AI 會怎麼做？（Lab 0）
0:20-0:35	Slurm 與 #SBATCH 參數（Lab 1、Lab 2）
0:35-0:50	陣列作業與相依作業（Lab 4、Lab 5）
0:50-1:10	自己改寫陣列作業（Lab 8）
1:10-1:20	效能分析 seff（Lab 7）
1:20-1:30	☕ 休息
1:30-2:10	讓 AI Agent 接手：多輪對話（Lab 10）
2:10-2:30	存成我的 skill（Lab 11）
2:30-2:55	用我的 skill 做質控並讀懂報告（第 04 章）
2:55-3:00	總結、預告第三堂

今天結束時，你會做到

1. 看懂 Slurm 腳本每一行 `#SBATCH` 的意思
2. 親手送出**標準、陣列、相依**三種作業，並用 `seff` 檢查資源
3. 用自然語言請 AI Agent 完成同樣的工作，並**用說的**糾正它
4. 把這些經驗存成**你自己的 skill**：`my-nano4-slurm`
5. 用自己的 skill 做 FASTQ 質控，並讀懂 MultiQC 報告

上課前：重新連線 🖐️

#	項目	完成了嗎？
1	在自己電腦啟動 ssh-proxy ，完成一次密碼與 OTP	☐
2	Antigravity 用 <code>Connect to Host...</code> 連上 nano4-proxy	☐
3	更新課程教材（指令如下）	☐
4	至少一個 AI Agent 已登入（Antigravity 內建 Agent 即可）	☐

```
cd "$HOME/Nano4-Docs"  
git pull --ff-only
```

本課程的固定設定（請記住）

計畫代號：GOV115088（國網生技醫藥高效能運算推廣與應用計畫）

佇列：ngs62g

規格：每個作業固定 -c 8 --mem=62G（國網官方規定，不可自行縮減）

環境：CPU-only，本次不使用 GPU

講義裡出現的 H200 / GB200 GPU、GOV113021、MST109178 等都是**參考資料**，照抄會被排程器拒絕。

第 03 章

Slurm 語法與作業調度

第 03 章： 10 個小節與 Lab 0-11

§	主題	對應 Lab	本堂課
8	暖身：沒有規則檔的 AI	 Lab 0	實作
1	為什麼需要 Slurm	—	講解
2	計畫、佇列與 QoS	 Lab 1	實作
3	#SBATCH 參數速查	 Lab 2	實作
4	Nodes、Tasks、CPUs、Memory	—	講解
5	陣列、相依、互動節點（GPU 參考）	 Lab 4、5、8、6	實作
6	監控與 <code>seff</code>	 Lab 7	實作
7	Apptainer 容器	 Lab 9	第三堂
9	讓 AI Agent 接手、我的 skill	 Lab 10、11	實作
10	常見問題	—	參考

 Lab 3 是 H200 GPU 作業，本課程跳過。



Lab 0 暖身：沒有規則檔時， AI 會怎麼做？（1/2）

刻意換到一個空白資料夾（沒有 AGENTS.md、CLAUDE.md、.agents/rules/）：

```
mkdir -p /work/$USER/slurm_quickstart
```

Antigravity 用 Open Folder 開啟 /work/<帳號>/slurm_quickstart，選一個 Agent 開新對話：

請協助建立並派送一個 FASTQ 生物資訊分析作業，工作資料夾就是目前開啟的這個資料夾：

1. 在 data/ 產生一個包含 1,000 條 reads 的測試用 FASTQ 檔 data/test_sample.fastq。
2. 在 script/ 寫一支 Python 腳本 script/fastq_qc_stats.py，統計這個 FASTQ 的 reads 數、平均讀長與 GC 含量 %。
3. 在 script/ 寫一支 Slurm 腳本：計畫 G0V115088、佇列 ngs62g，每個作業申請 8 核心、62G 記憶體，在計算節點執行上面的 Python 腳本。
4. 送出前先告訴我你的計畫；送出後回報 Job ID，跑完把分析過程與結果寫成 report.md。



Lab 0 暖身：對照 AGENTS.md 檢查（2/2）

檢查項目	✓ / ✗
<code>--account=G0V115088</code> 、 <code>--partition=ngs62g</code> 、 <code>--cpus-per-task=8</code> 、 <code>--mem=62G</code>	
執行內容第一行是 <code>module purge</code>	
日誌用 <code>%x-%j.out</code> / <code>%x-%j.err</code> ；若寫到 <code>logs/</code> 這類子目錄，送出前有先建立	
Python 分析在計算節點上執行，不是在登入節點直接跑	
送出後沒有在迴圈中反覆查詢 <code>squeue</code>	
<code>report.md</code> 的 reads 數等於 1000	

📌 記下 ✗ 的項目：Lab 10 會在放了 `AGENTS.md` 的資料夾重做，比較有沒有規則檔的差別。

§1 為什麼需要 Slurm ?

數百位研究者同時使用數百台計算節點；沒有排程，多人同時跑重度運算會讓記憶體耗盡、機器癱瘓。

Slurm 做的事	意思
佇列調度	依計畫配額與優先權分配節點與時間
資源隔離	你申請的 8 核心、62 GB 完全歸你，不受別人干擾
作業計費	依使用的核心時間計算服務點數（SU）

⚠️ Antigraity 的終端機在**登入節點**：這裡只寫腳本、送作業、看日誌，運算交給 `sbatch`。

§2-A 查詢計畫錢包與授權

送出作業前，先確認計畫有點數、也有排程權限：

```
# 1. 查詢名下所有計畫的點數餘額  
wallet
```

```
# 2. 查詢特定計畫餘額 ( 速度較快 )  
wallet GOV115088
```

```
# 3. 查詢自己帳號在 Slurm 排程系統中綁定的授權 (Association)  
sacctmgr -nP show assoc user="$(whoami)" format=Account,Partition,QOS
```

§2-B 計畫與佇列：不能混用

計畫類別	例子	可以用的佇列
本課程	GOV115088	只有 <code>ngs62g</code> （每作業 8 核心 / 62 GB，最長 4 天）
一般 AI 計畫（參考）	GOV113021	H200 dev、8gpu...；GB200 gb200-dev...
生醫平台計畫（參考）	MST109178、ENT109430	所有 <code>ngs*</code> CPU、大記憶體與生醫 GPU 佇列

- GOV115088 只列在 `ngs62g` 的 `AllowAccounts`；送到 `ngstest`、`ngs32g`、`ngscourse*` 會被拒絕
- 派送前可用 `scontrol show partition <PARTITION>` 檢查 `AllowAccounts`

§2-C ngs62g 規格與最常見的錯誤

佇列	核心 -c	記憶體 --mem	最長時間	用途
ngs62g	8	62 GB (固定)	4 天 (96h)	本課程： FASTQ QC 、擴增子分析

⚠ **漏寫 --mem**：Slurm 會預設申請整台節點約 1 TB 記憶體，超過 QoS 上限，作業會被拒絕或一直卡在 (QOSMaxMemoryPerJob)。

✅ ngs62g 作業一定要明確寫出 --cpus-per-task=8 與 --mem=62G。



Lab 1：查詢錢包與可用的排程分區

```
cd "$HOME/Nano4-Docs/03-slurm-syntax-and-job-management"
```

```
wallet
```

```
./scripts/slurm_status.sh
```

```
scontrol show partition ngs62g
```

請確認： `wallet` 列出 `GOV115088`；`ngs62g` 的 `AllowAccounts` 包含 `gov115088`。

§3 官方範例：ngs62g 的 Slurm 腳本

```
#!/usr/bin/sh
#SBATCH -A GOV115088      # Account name/project number (官方範例為 MST109178)
#SBATCH -J Job_name       # Job name
#SBATCH -p ngs62g         # Partition Name 等同 PBS 裡面的 -q Queue name
#SBATCH -c 8              # 使用的 core 數 請參考 Queue 資源設定
#SBATCH --mem=62g         # 使用的記憶體量 請參考 Queue 資源設定
#SBATCH -o out.log        # Path to the standard output file
#SBATCH -e err.log        # Path to the standard error output file
#SBATCH --mail-user=XXXX@nlar.org.tw # email
#SBATCH --mail-type=BEGIN,END # 指定送出 email 時機 可為 NONE, BEGIN, END, FAIL, REQUEUE, ALL

echo 'Hello world!' ## 這邊寫入你要執行的指令
```

本課程範本另外做兩處調整：日誌改用 `%x-%j.out` / `%x-%j.err`；執行內容第一行加入 `module purge`。

§3 #SBATCH 參數速查

參數	簡寫	功能	本課程範例
<code>--account=<ID></code>	<code>-A</code>	計費計畫代號	<code>--account=G0V115088</code>
<code>--job-name=<NAME></code>	<code>-J</code>	作業名稱	<code>--job-name=fastq_qc</code>
<code>--partition=<NAME></code>	<code>-p</code>	排程分區	<code>--partition=ngs62g</code>
<code>--cpus-per-task=<N></code>	<code>-c</code>	每個行程的核心數	<code>--cpus-per-task=8</code>
<code>--mem=<SIZE></code>		必填： 總記憶體	<code>--mem=62G</code>
<code>--time=<D-HH:MM:SS></code>	<code>-t</code>	執行時間上限	<code>--time=00:30:00</code>
<code>--output=<FILE></code>	<code>-o</code>	標準輸出日誌	<code>--output=%x-%j.out</code>
<code>--error=<FILE></code>	<code>-e</code>	標準錯誤日誌	<code>--error=%x-%j.err</code>

 日誌檔名：`%x` = 作業名稱、`%j` = Job ID、`%A` = 陣列主 ID、`%a` = 子任務序號

§4 資源四要素： Nodes、Tasks、CPUs、Memory

```
1 個實體節點    --nodes=1
└─ 1 個作業行程  --ntasks-per-node=1
    └─ 8 個核心   --cpus-per-task=8
62 GB 記憶體    --mem=62G
```

單機多執行緒程式（FastQC、Python multiprocessing）的標準寫法：

```
#SBATCH --account=G0V115088
#SBATCH --partition=ngs62g
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=62G
```



Lab 2：提交第一個標準 CPU 作業

```
cat templates/standard_cpu_job.slurm
sbatch --account=G0V115088 templates/standard_cpu_job.slurm
queue -u $(whoami)
ls -la cpu_job-*.out
cat cpu_job-*.out
```

- `queue` 可能先顯示 `PD`（排隊），接著變成 `R`（執行中）
- 作業結束後會從清單中消失，這時就可以看日誌

§5-A 陣列作業：一次送出多個相同任務

有 10 個 FASTQ 要處理時，不必寫 10 份腳本：

```
#SBATCH --partition=ngs62g
#SBATCH --cpus-per-task=8
#SBATCH --mem=62G
#SBATCH --array=1-10%4          # 總共 10 個任務，%4 代表最多同時平行執行 4 個
#SBATCH --output=%x-%A_%a.out # %A 為主作業 ID，%a 為子任務序號 (1~10)

# 在腳本中透過 ${SLURM_ARRAY_TASK_ID} 取得當前處理序號
SAMPLE=$(sed -n "${SLURM_ARRAY_TASK_ID}p" sample_list.txt)
echo "正在處理樣本： ${SAMPLE}"
```



Lab 4：陣列作業演練

```
sbatch --account=G0V115088 templates/array_job.slurm  
queue -u $(whoami)  
ls -la array_job-*.out
```



作業約 17 秒就跑完，`queue` 不一定看得到。用下面的指令看開始時間分成 4、4、2 三批，就是 `%4` 的效果：

```
sacct -j <JOB_ID> -X -o JobID,Start,End
```

§5-B 相依作業：前一步成功才開始下一步

```
# 步驟 1：送出資料前處理任務 (--parsable 只輸出 Job ID)
JOB1=$(sbatch --parsable --account=G0V115088 stage1_preprocess.slurm)

# 步驟 2：只有在 JOB1 成功結束 (afterok) 時，才啟動步驟 2
JOB2=$(sbatch --parsable --dependency=afterok:${JOB1} --account=G0V115088 stage2_qc.slurm)

# 步驟 3：步驟 2 完成後，自動產出統計報告
sbatch --dependency=afterok:${JOB2} --account=G0V115088 stage3_report.slurm
```

 `stage*_*.slurm` 是示意檔名； Lab 5 用同一支範本代表三個階段。



Lab 5：相依流水線串接

```
bash templates/workflow_dependency.sh  
queue -u $(whoami)
```

✓ 後面兩個階段的狀態會顯示 (Dependency)：正在等前一個階段完成



Lab 8：自己改寫陣列作業（1/2）

目標：每個子任務處理一個 FASTQ 樣本（第 04 章附的 4 個樣本），計算 read 數。

```
cp templates/array_job.slurm my_array_fastq.slurm
```

用 Antigravity 打開 `my_array_fastq.slurm`，改三處：

1. `--array=1-10%4` 改成 `--array=1-4`（4 個樣本 → 4 個子任務）
2. 作業名稱改成 `array_fastq`
3. 把 `sleep 5` 那段換成：依 `${SLURM_ARRAY_TASK_ID}` 挑出第 N 個 FASTQ，計算 read 數並印出



Lab 8：提示與預期結果（2/2）

```
FASTQ_DIR="$HOME/Nano4-Docs/04-ai-assisted-bio-pipeline/demo_data/fastq_raw"
mapfile -t FASTQ_FILES < <(ls "${FASTQ_DIR}"/*.fastq.gz)
FASTQ="${FASTQ_FILES[$((SLURM_ARRAY_TASK_ID - 1))]}" # 陣列從 0 開始，所以要減 1
READS=$(( $(gzip -dc "${FASTQ}" | wc -l) / 4 ))      # FASTQ 每 4 行是一條 read
```

```
sbatch my_array_fastq.slurm
cat array_fastq-*_*.out
```

子任務 1		節點 25a-cpn01		樣本 sample_01_R1.fastq.gz		reads = 1000
子任務 2		節點 25a-cpn01		樣本 sample_02_R1.fastq.gz		reads = 1000
...						

§5-D + Lab 6：互動式計算節點（選做）

不想每改一行就 `sbatch` 一次時，直接「走進」計算節點測試：

```
./scripts/interactive_salloc.sh G0V115088 ngs62g 8 00:10:00 62G
```

```
hostname      # 應顯示 25a-cpn*  
free -h  
exit          # 用完一定要離開，釋放資源
```

 `salloc` 用完一定要 `exit`，否則會一直占用資源並計費。

§5-C / §5-E GPU 與跨節點運算（參考）

- H200 GPU 作業需要一般 AI 計畫（例如 `GOV113021`），並加上 `--gres=gpu:1`
- 跨節點不一定比較快：小任務時間都花在節點之間的通訊
- 能放進單一節點的工作，優先用單節點

✗ 本課程是 **CPU-only**：不申請 GPU、不使用 `dev` 或 `ngs1gpu` ~ `ngs8gpu`，Lab 3 跳過。

§6-A 常用排程管理指令

目的	指令
提交作業	<code>sbatch job.slurm</code>
查看自己的作業	<code>squeue -u \$(whoami)</code>
取消作業	<code>scancel <JOB_ID></code>
查看作業詳細資訊	<code>scontrol show job <JOB_ID></code>
查詢已結束作業	<code>sacct -j <JOB_ID> --format=JobID,JobName,State,Elapsed,MaxRSS</code>
查看分區空間狀態	<code>sinfo -p ngs62g</code>

⚠ **不要用 `watch` 或迴圈反覆查詢 `squeue`：**會加重排程系統負擔。需要通知時改用 `#SBATCH --mail-type=END,FAIL`。

§6-B 作業狀態代碼

代碼	狀態	意思
PD	PENDING	排隊中，看 <code>NODELIST(REASON)</code> 了解原因
R	RUNNING	正在計算節點上執行
CD	COMPLETED	正常結束，ExitCode 為 0
F	FAILED	程式出錯，查看 <code>%x-%j.err</code>
TO	TIMEOUT	超過 <code>--time</code> 上限
CA	CANCELLED	被 <code>scancel</code> 取消
NF	NODE_FAIL	節點硬體故障，不是你的問題，請回報國網

§6-C seff：看資源有沒有用對

```
seff <JOB_ID>
```

```
Job ID: 1073764  
State: COMPLETED (exit code 0)  
Nodes: 1  
Cores per node: 8  
CPU Utilized: 00:06:24  
CPU Efficiency: 80.00% of 00:08:00 core-walltime  
Memory Utilized: 2.15 GB  
Memory Efficiency: 3.47% of 62.00 GB
```

- 記憶體效率低在 `ngs62g` 是正常的，**不要**因此調降 `--mem`
- 重點看 **CPU 效率**：8 核心有沒有被用滿（例如 `fastqc -t ${SLURM_CPUS_PER_TASK}`）
- `ExitCode 137` = 記憶體不足：減少單一作業的資料量，或拆成陣列作業



Lab 7：用 `saff` 診斷作業

對前面任何一個已完成的 Job ID 執行：

```
saff <JOB_ID>
```

請回答： CPU Efficiency 是多少？這個作業有沒有把 8 核心用滿？



Lab 2 的範本只有 `sleep 5`，CPU 效率很低是正常的；真正的分析程式要用 `$ {SLURM_CPUS_PER_TASK}` 把核心用滿。

§7 Apptainer 容器（第三堂 Lab 9 實作）

超級電腦上不能用 Docker（需要 root）；**Apptainer**（Singularity）可以在沒有 root 的情況下執行容器：

```
singularity pull /work/${USER}/ubuntu_22.04.sif docker://ubuntu:22.04
singularity exec -B /work/${USER}:/mnt /work/${USER}/ubuntu_22.04.sif ls /mnt # 在容器內列出掛載的 /work 目錄
```

- Nano4 計算節點可以直接連外網，從 Docker Hub 下載映像不需要 Proxy
- Lab 9 需要第 04 章產生的 FastQC 結果，放到第三堂做

休息 10 分鐘

回來後讓 AI Agent 接手，把剛才學到的東西存成你自己的 skill。

§9 讓 AI Agent 接手：多輪對話與我的 skill

剛才你親手送出了標準、陣列與相依作業；現在請 AI Agent 用自然語言完成同樣的工作：

- 你只描述想做的分析，Agent 負責寫腳本、送作業、解讀結果
- 你負責檢查它做得對不對，做錯時**用說的**糾正
- 最後請 Agent 把經驗存成你自己的 skill

 在教材以外的 `/work/$USER/slurm_lab` 進行：Agent 看不到課程提供的 Skills，做出來的 skill 才是你自己的經驗。



Lab 10：用自然語言多輪操作 Slurm （1/3）

建立工作資料夾（放入 AGENTS.md、CLAUDE.md、.agents/rules/nano4.md 與 4 個 FASTQ）：

```
bash "$HOME/Nano4-Docs/03-slurm-syntax-and-job-management/scripts/setup_agent_workspace.sh"
```

Antigravity 開啟 /work/<帳號>/slurm_lab，開新對話，**第 1 輪：陣列作業**

請先閱讀 AGENTS.md。fastq_raw/ 裡有 4 個定序樣本，請幫我算出每個樣本各有幾條 reads。每個樣本用一個子任務平行處理。送出前先告訴我你的計畫。



Lab 10：第 2-4 輪（2/3）

第 2 輪：相依作業

接著請對這 4 個樣本做 FastQC 品質檢查；全部檢查完成後，再自動用 MultiQC 彙整成一份報告。
第二步要等第一步成功才開始，結果放在 qc/ 資料夾。

第 3 輪：資源檢查

剛才的作業都跑完了嗎？請用 seff 看看資源用得如何，用白話告訴我，並說明這 4 個樣本的品質結果。

第 4 輪：故意提出錯誤要求

為了省點數，請把記憶體改成 16G、核心改成 2 顆，再送一次 FastQC。



Lab 10：逐輪檢查（3/3）

檢查項目	第 1 輪	第 2 輪	第 3 輪	第 4 輪
先說明計畫、等你同意才送出				
G0V115088 + ngs62g + -c 8 --mem=62G				
module purge ; FastQC 有載入 biology/JDK/26.0.1				
用 sbatch 送到計算節點				
相依作業用 --dependency=afterok	—		—	—
沒有反覆查詢 squeue				
用白話解讀結果				—

⚠ **第 4 輪的正確反應是拒絕：** ngs62g 依規定必須固定 -c 8 --mem=62G 。

💡 Agent 做錯時不要自己改腳本，**用說的**告訴它哪裡錯、為什麼錯；這些糾正就是 Lab 11 要存的經驗。



Lab 11：把經驗存成我的 skill（1/3）

Skill 是一個資料夾，裡面的 `SKILL.md` 寫著「什麼時候用、怎麼做、要注意什麼」。在 Lab 10 的**同一個對話**中輸入：

請把我們這次對話的經驗整理成一個 skill，名稱是 `my-nano4-slurm`，存到 `~/.agents/skills/my-nano4-slurm/SKILL.md`。

`SKILL.md` 開頭要有 `name` 與 `description` 的 YAML frontmatter；`description` 要寫清楚什麼時候該使用這個 skill。

內容請包含：

1. 本課程的計畫、佇列與固定規格（`GOV115088`、`ngs62g`、`-c 8 --mem=62G`）
2. 單一作業、陣列作業、相依作業三種 Slurm 腳本範本（放在同一個資料夾的 `templates/`）
3. 送出前的檢查清單，以及 `sbatch --test-only` 不會檢查規格這件事
4. 這次對話中你犯過、被我糾正過的錯誤，以及正確做法
5. 作業跑完後如何確認結果、如何用白話回報



Lab 11：檢查並安裝給三個 Agent（2/3）

1. 打開 `~/.agents/skills/my-nano4-slurm/SKILL.md` 讀一遍：有沒有寫進 Lab 10 糾正過的錯誤？
2. 複製給三個 Agent：

```
bash "$HOME/Nano4-Docs/03-slurm-syntax-and-job-management/scripts/install_my_skill.sh"
```

- ✓ `/home/<帳號>/~/.claude/skills/my-nano4-slurm`
- ✓ `/home/<帳號>/~/.gemini/config/skills/my-nano4-slurm`
- 👉 三個 Agent 都要開一個新對話，才會讀到更新後的 skill。

Agent	讀取位置
Codex	<code>~/.agents/skills/</code>
Claude Code	<code>~/.claude/skills/</code>
Antigravity 內建 Agent	<code>~/.gemini/config/skills/</code>



Lab 11：換一個 Agent 驗收（3/3）

在**另一個 Agent** 開新對話（例如 Lab 10 用 Antigravity，這裡改用 Codex 或 Claude Code）：

你有哪些 skills？請用 my-nano4-slurm 說明，如果我要對 20 個 FASTQ 樣本做品質檢查，你會怎麼安排 Slurm 作業（先不要送出）。

✓ Agent 會提到 my-nano4-slurm，並依你的 skill 規劃 ngs62g、-c 8 --mem=62G、陣列加相依作業，不必你再提醒

📌 Skill 是純文字檔，之後可以隨時請 Agent 補充，改完再執行一次 install_my_skill.sh。

§10 第 03 章常見問題

狀況	原因與解法
Invalid account or account/partition combination specified	計畫沒有該佇列權限；本課程只能用 <code>ngs62g</code>
一直 PD，原因是 (QOSMaxMemoryPerJob)	漏寫 <code>--mem</code> ；加上 <code>--cpus-per-task=8</code> 與 <code>--mem=62G</code>
作業瞬間失敗， <code>_open_output_file: No such file or directory</code>	日誌寫到不存在的 <code>logs/</code> ；先 <code>mkdir -p logs</code> 或改用 <code>%x-%j.out</code>
<code>sbatch --test-only</code> 通過，送出後卻卡住	<code>--test-only</code> 不檢查 QoS 與規格，要自己對照

第 04 章

AI 輔助生醫質控管線

第 04 章： 8 個小節

§	主題	本堂課
1	FASTQ、FastQC 與 MultiQC	講解
2	用我的 skill 請 AI Agent 做質控	實作（主線）
3	請 AI 寫分析腳本的 prompt 技巧	參考
4	檔案結構	參考
5	對照組：在登入節點手動執行	第三堂 Lab 9 前再跑
6	檢視互動式報告	實作
7	 練習：讀懂 MultiQC 報告	實作
8	登入節點的限制	講解

§1 FASTQ、FastQC 與 MultiQC

名詞	是什麼
FASTQ	定序儀輸出的檔案；每條 read 4 行：@Header、序列、+、品質分數
FastQC	檢查單一樣本的品質：鹼基品質（Q30 = 99.9% 準確）、ATCG 比例、GC 含量、接頭殘留
MultiQC	把所有樣本的 FastQC 結果彙整成一份互動式網頁報告

 資料 → 工具 → 報告：這個模式在各種研究領域都通用。

§2 用我的 skill 請 AI 做質控（1/2）

Antigravity 開啟 `/work/<帳號>/slurm_lab`，**開新對話**（新對話才會讀到最新的 skill）：

請使用 `my-nano4-slurm skill`。`fastq_raw/` 裡是 4 個 16S 定序樣本，我想知道它們的定序品質好不好、適不適合做後續分析。

請用 FastQC 檢查每個樣本，再用 MultiQC 彙整成一份報告，結果放在 `qc_skill/`。
送出前先告訴我你的計畫；跑完後告訴我報告的位置，並用白話說明：每個樣本有幾條 reads、品質分數如何、有沒有需要注意的警告。

👉 你沒有打任何指令，只說了想知道的生物學問題

§2 這次 Agent 不需提醒就做對了嗎？（2/2）

檢查項目	✓ / ✗
先說明計畫，等你同意才送出	
<code>GOV115088 + ngs62g + -c 8 --mem=62G</code>	
<code>module purge</code> ，FastQC 一起載入 <code>biology/JDK/26.0.1</code>	
用 <code>sbatch</code> 送到計算節點；確認 FastQC 報告數量等於樣本數	
用白話回報結果與報告位置	

💡 還有 ✗：請 Agent 說明原因並修正，再請它把教訓補進 **my-nano4-slurm**，重新執行 `install_my_skill.sh`。skill 就是這樣越用越少犯錯。

§3 請 AI 寫分析腳本：好 prompt 的四個要素

1. 角色與環境：「你熟悉生物資訊與 Nano4 HPC 環境」
 2. 資料在哪、結果放哪：輸入資料夾與輸出資料夾寫清楚
 3. 工具與版本：`module load biology/JDK/26.0.1 biology/FastQC/0.11.9 biology/MultiQC`
 4. 錯誤處理與回報：找不到工具要停止、跑完要說報告在哪
- 📌 完整範例見講義第 04 章 §3 與 `prompts/ai_prompt_bio_pipeline.md`；有了自己的 skill 後，prompt 就可以短很多。

§4 本章的檔案結構

```
04-ai-assisted-bio-pipeline/  
├── README.md  
├── prompts/  
│   └── ai_prompt_bio_pipeline.md  
├── scripts/  
│   ├── download_demo_fastq.sh  
│   ├── run_fastqc_multiqc.sh  
│   └── view_multiqc_report.sh  
└── demo_data/  
    ├── fastq_raw/  
    ├── fastqc_out/  
    └── multiqc_out/
```

本章完整教學手冊

AI Agent 引導提示詞範本

[1] 自動準備 / 取樣 4 組示範 FASTQ 檔案

[2] 執行 FastQC 分析與 MultiQC 聚合

[3] 啟動 HTTP 服務並引導 VS Code 轉送預覽

資料與報告存放區

原始 FASTQ 檔案 (*.fastq.gz)

FastQC 輸出產物 (*_fastqc.zip)

MultiQC 匯總產物 (multiqc_report.html)

§5 對照組：在登入節點手動執行

不經過 AI、也不經過 Slurm 的手動版本；4 個微型樣本在登入節點只需數秒：

```
cd "$HOME/Nano4-Docs/04-ai-assisted-bio-pipeline/scripts"  
bash download_demo_fastq.sh  
bash run_fastqc_multiqc.sh
```

- 產生 `demo_data/fastqc_out/`，第三堂 Lab 9（容器）會用到
- 只有這種微型資料才能在登入節點跑；真實資料一律用 Slurm

§6 打開 MultiQC 報告

方式 A：連接埠轉送

```
cd "$HOME/Nano4-Docs/04-ai-assisted-bio-pipeline/scripts"
bash view_multiqc_report.sh # 第 5 節對照組的報告
bash view_multiqc_report.sh /work/$USER/slurm_lab/qc_skill/<報告所在資料夾> # 第 2 節 Agent 產生的報告
```

- 依畫面提示在瀏覽器打開 `http://localhost:<埠號>/multiqc_report.html`
- 看完執行 `tmux kill-session -t svc-multiqc-report` 關閉預覽

方式 B：下載到筆電：檔案總管找到 `multiqc_report.html` → 按右鍵 → **Download...**



§7 練習：讀懂 MultiQC 報告（5-10 分鐘）

1. **General Statistics**：4 個樣本各有幾條 reads？GC 含量大約多少？
2. **Sequence Length Distribution**：reads 長度範圍？為什麼出現黃色警告？
3. **Per Base Sequence Quality**：品質分數落在哪個顏色區域？
4. **Status Checks**：哪個樣本的哪一項是紅色（fail）？其他樣本同一項是什麼顏色？
5. 綜合判斷：第 4 題的紅色代表這個樣本不能用嗎？



也可以問 Agent：「sample_04 的 Per Sequence GC Content 為什麼是紅色？」請它用報告中的數據回答，你再對照圖表確認。

§8 為什麼真實資料一定要用 Slurm ?

微型練習資料	真實研究資料
4 個樣本、各 1000 reads	數十到數千個樣本
每個檔案約 80 KB	每個檔案數百 MB 到數十 GB
登入節點數秒完成	需要數十核心、數百 GB 記憶體

⚠ 登入節點上超過 5 分鐘的重度運算會被系統**直接清除**。

👉 這就是為什麼第 2 節一開始就請 AI Agent 把分析寫成 Slurm 作業，送到 `ngs62g` 計算節點。

今天學到了什麼

主題	重點
Slurm 腳本	G0V115088 → ngs62g → -c 8 --mem=62G ; module purge ; %x-%j.out
三種作業	標準 sbatch、陣列 --array、相依 --dependency=afterok
資源檢查	seff：看 CPU 效率，不要調小 --mem
AI Agent	沒有規則檔會出錯；有 AGENTS.md 會好很多；做錯時用說的糾正
我的 skill	my-nano4-slurm 存在 ~/.agents/skills/，用 install_my_skill.sh 給三個 Agent
生醫質控	用 skill 做 FastQC / MultiQC，讀懂紅黃綠代表的意思

下課前檢核

- ☐ 1. Lab 8 的 4 個日誌各自對應一個樣本，reads 都是 1000
- ☐ 2. `~/ .agents/skills/my-nano4-slurm/SKILL.md` 存在，`install_my_skill.sh` 顯示兩個 
- ☐ 3. 在瀏覽器打開 Agent 產生的 MultiQC 報告，回答了 S7 的 5 個問題
- ☐ 4.  離開前在 proxy 視窗按 `Ctrl+C` (尤其是共用電腦)

課後自學（選做）

章節	內容
第 03 章 Lab 6	用 <code>salloc</code> 進入互動式計算節點
第 03 章 §5-C / §5-E	GPU 作業與跨節點運算（參考）
第 04 章 §3	自己寫一段完整的分析 prompt，和 skill 版本比較
我的 skill	把今天遇到的錯誤補進 <code>my-nano4-slurm</code> ，重新執行 <code>install_my_skill.sh</code>

下一堂預告

第三堂：進階與真實案例

- 開課先送出 nf-core/ampliseq 官方測試（約 28 分鐘）
- 第 05 章：請 AI 把流程改寫成離線與外網直連兩種 Slurm 作業
- 第 06 章：用課程 Skills 抓錯，並和**你的 skill** 比較、補強
- 第 07 章：解讀真實 16S 腸道菌相分析結果

📖 講義：<https://gemini960114.github.io/Nano4-Docs/>

謝謝大家！ 🙌